

Resume Maker

AI-Powered Resume & Cover Letter Generator

Complete Application Documentation

Version 1.0.0

September 2026

Table of Contents

1. Overview
2. Features
3. Architecture & Project Structure
4. Tech Stack & Dependencies
5. Setup & Installation
6. Configuration (Environment Variables)
7. API Endpoints
8. Self-Hosting Guide (Docker)
9. Security Features
10. Health Monitoring
11. Template Customization
12. Troubleshooting

1. Overview

Resume Maker is an AI-powered web application that generates ATS-optimized resumes and personalized cover letters. Users fill in their details, optionally provide a job description (via URL or pasted text), and the app uses the **Groq API** (free Llama 3 / OpenAI OSS 120B model) to tailor the documents to the specific role.

The generated documents use clean, single-column HTML templates that parse well with Applicant Tracking Systems (ATS), and can be downloaded as professional PDFs.

> **Key advantage:** The Groq API is completely free — no credit card required. It provides access to OpenAI OSS 120B (comparable to GPT-4) with fast inference speeds.

2. Features

Feature	Description
CV Import	Upload an existing PDF/DOCX/TXT resume; AI parses it and pre-fills the form
Job Description Input	Paste a job posting URL or paste the full job description text directly
AI-Tailored Resumes	AI rewrites summary and experience with keywords from the job description
Cover Letter Generation	Personalized cover letters matched to the specific role
ATS-Friendly Templates	Single-column, clean HTML templates optimized for ATS parsers
PDF Export	Download individual or combined PDFs (resume + cover letter)
GitHub Integration	Fetch GitHub repos and add selected projects to your resume
Academic Document Parsing	Upload transcripts to auto-fill education fields
Self-Hosting Ready	Docker containerization with Nginx reverse proxy
Production Security	Helmet headers, rate limiting, secure sessions, health checks

3. Architecture & Project Structure

```
resume-maker/
├── server.js # Express server & all routes
├── package.json # Dependencies & scripts
├── Dockerfile # Multi-stage Docker build
├── docker-compose.yml # Docker Compose orchestration
├── .env # Environment variables (not committed)
├── .env.example # Environment template
├── .env.production # Production environment template
├── .dockerignore # Docker build exclusions
├── nginx/
│ ├── nginx.conf # Main Nginx config (gzip, rate limiting, security headers)
│ └── conf.d/
│ └── default.conf # Server blocks (HTTP + HTTPS template)
├── lib/
│ ├── ai.js # Groq/Llama 3 AI integration
│ ├── scraper.js # Job description URL scraper
│ ├── templates.js # ATS-friendly HTML resume & cover letter templates
│ ├── pdf.js # PDF generation with Puppeteer
│ ├── document-parser.js # PDF/DOCX text extraction & AI parsing
│ └── github.js # GitHub API integration (repos, READMEs)
├── public/
│ ├── index.html # Main input form
│ ├── preview.html # Side-by-side resume/cover letter preview
│ ├── css/style.css # Application styles
│ └── js/app.js # Client-side JavaScript (form handling, uploads, GitHub)
├── output/ # Temporary PDF files (auto-cleaned after download)
├── uploads/ # Uploaded documents (auto-cleaned after parsing)
└── docs/
└── documentation.html # This document
```

Request Flow

1. User fills in personal info, education, experience, and skills on the form
2. User provides a job description (URL or pasted text)
3. On submit, the server scrapes the URL (if provided) or uses the pasted text
4. The AI (Groq API) generates a tailored resume and cover letter
5. User previews both documents side-by-side
6. User downloads individual or combined PDFs

4. Tech Stack & Dependencies

Runtime

Technology	Version	Purpose
Node.js	18+	JavaScript runtime
Express	^4.18.0	Web framework & routing
express-session	^1.18.0	Server-side session management
helmet	^8.3.0	HTTP security headers
express-rate-limit	^8.7.0	Request rate limiting
dotenv	^16.4.0	Environment variable loading

AI & Processing

Technology	Version	Purpose
OpenAI SDK	^4.50.0	Groq API client (OpenAI-compatible)
Puppeteer	^25.9.0	PDF generation via headless Chromium
pdf-parse	^2.4.5	Extract text from uploaded PDFs
mammoth	^1.12.2	Extract text from uploaded DOCX files

Web Scraping & HTTP

Technology	Version	Purpose
axios	^1.7.0	HTTP client for scraping & GitHub API
cheerio	^1.0.0	HTML parsing (jQuery-like) for job scraping
multer	^2.3.0	File upload handling (multipart/form-data)

Dev Dependencies

Technology	Version	Purpose
nodemon	^3.1.0	Auto-restart server on file changes

5. Setup & Installation

Prerequisites

- Node.js v18 or higher
- A free Groq API key from `console.groq.com`
- npm (comes with Node.js)

Development Setup

```
# Clone the repository
git clone https://github.com/your-username/resume-maker.git
cd resume-maker

# Install dependencies
npm install

# Create environment file
cp .env.example .env

# Edit .env and add your Groq API key
# GROQ_API_KEY=gsk_your_key_here

# Start development server (with auto-reload)
npm run dev

# Or start without auto-reload
npm start
```

Open `http://localhost:3000` in your browser.

> **Getting a Groq API Key:** Visit `console.groq.com`, sign up (free, no credit card), create an API key, and paste it into your `.env` file.

6. Configuration (Environment Variables)

Variable	Description	Default	Required
`GROQ_API_KEY`	Your Groq API key for AI generation	—	Yes
`PORT`	Server port number	3000	No

`SESSION_SECRET`	Secret for session encryption	—	Yes (production)
`NODE_ENV`	Environment mode: `development` or `production`	development	No
`TRUST_PROXY`	Set to `1` when behind Nginx reverse proxy	—	No
`RATE_LIMIT_WINDOW_MS`	Rate limit window in milliseconds	900000 (15 min)	No
`RATE_LIMIT_MAX_REQUESTS`	Max requests per IP per window	100	No

Generating a Secure Session Secret

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

.env.example

```
# Groq API Key (free) - get yours at https://console.groq.com
GROQ_API_KEY=your_groq_api_key_here

# Server Port
PORT=3000

# Session Secret
SESSION_SECRET=your_session_secret_here

# Node Environment
NODE_ENV=development

# Trust proxy (set to 1 behind Nginx)
# TRUST_PROXY=1
```

7. API Endpoints

Method	Path	Description
`GET`	`/`	Home page — input form
`POST`	`/generate`	Process form → generate resume + cover letter
`GET`	`/preview`	Side-by-side preview of resume & cover letter
`GET`	`/health`	Health check endpoint (JSON response)
API Endpoints		
`GET`	`/api/resume-html`	Get rendered resume HTML
`GET`	`/api/cover-letter-html`	Get rendered cover letter HTML
`POST`	`/api/parse-academic`	Upload academic document → extract education info
`POST`	`/api/parse-cv`	Upload existing CV → extract all structured data
`GET`	`/api/github/repos/:username`	Fetch public GitHub repositories for a user
`POST`	`/api/github/build-projects`	Build formatted projects text from selected repos
Download Endpoints		
`GET`	`/download/resume`	Download resume as PDF
`GET`	`/download/cover-letter`	Download cover letter as PDF
`GET`	`/download/both`	Download both PDFs sequentially

POST /generate — Request Body

Field	Type	Required	Description
`full_name`	string	Yes	Full name
`email`	string	Yes	Email address
`phone`	string	Yes	Phone number
`location`	string	No	City, State/Country

`linkedin`	string	No	LinkedIn profile URL
`summary`	string	No	Professional summary
`job_url`	string	No	Job posting URL to scrape
`job_text`	string	No	Pasted job description text (preferred if URL fails)
`edu_school[]`	string[]	Yes	School/university names
`edu_degree[]`	string[]	Yes	Degree types
`edu_field[]`	string[]	No	Fields of study
`edu_gpa[]`	string[]	No	GPAs
`edu_start[]`	string[]	No	Start dates
`edu_end[]`	string[]	No	End dates
`exp_company[]`	string[]	Yes	Company names
`exp_title[]`	string[]	Yes	Job titles
`exp_start[]`	string[]	No	Start dates
`exp_end[]`	string[]	No	End dates
`exp_description[]`	string[]	No	Role descriptions
`skills`	string	No	Comma-separated skills
`certifications`	string	No	Certifications
`projects`	string	No	Projects description

GET /health — Response

```
{
  "status": "ok",
  "timestamp": "2026-09-12T00:00:00.000Z",
  "uptime": 12345.678,
  "memoryUsage": { "rss": ..., "heapTotal": ..., "heapUsed": ... },
  "cpuUsage": { "user": ..., "system": ... },
  "environment": "production",
  "version": "1.0.0"
}
```

8. Self-Hosting Guide (Docker)

Option 1: Docker Compose (Recommended)

```
# 1. Clone and enter the project
git clone https://github.com/your-username/resume-maker.git
cd resume-maker

# 2. Create environment file
cp .env.production .env
# Edit .env with your GROQ_API_KEY and SESSION_SECRET

# 3. Start everything (app + Nginx reverse proxy)
docker-compose up -d

# 4. Verify
docker-compose ps
curl http://localhost/health

# 5. View logs
docker-compose logs -f
```

The app will be accessible at:

- **HTTP:** `http://your-server-ip` (port 80 via Nginx)
- **Direct:** `http://your-server-ip:3000` (bypassing Nginx)
- **Health Check:** `http://your-server-ip/health`

Option 2: Docker Only (No Nginx)

```
# Build
docker build -t resume-maker .

# Run
docker run -d \
  --name resume-maker \
  -p 3000:3000 \
  -e GROQ_API_KEY=your_key \
  -e NODE_ENV=production \
  -e SESSION_SECRET=your_secret \
  resume-maker
```

Option 3: Manual Deployment (No Docker)

```
# Install production dependencies
npm ci --only=production

# Set environment variables
export NODE_ENV=production
export GROQ_API_KEY=your_key
export SESSION_SECRET=your_secret

# Start
node server.js
```

SSL/TLS (HTTPS)

1. Obtain SSL certificates (e.g., Let's Encrypt with Certbot)
2. Place them in `nginx/ssl/` as `fullchain.pem` and `privkey.pem`
3. Uncomment the HTTPS server block in `nginx/conf.d/default.conf`
4. Update `server_name` to your domain
5. Restart: `docker-compose restart nginx`

9. Security Features

Feature	Implementation	Details
Security Headers	Helmet.js	X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, CSP, HSTS, and more
Rate Limiting	express-rate-limit	General: 100 req/15min. API: 50 req/15min. Configurable via env vars.
Secure Sessions	express-session	HTTP-only cookies, secure flag in production, 24h expiry
Trust Proxy	Express setting	Supports X-Forwarded-For when behind Nginx
Non-root Container	Dockerfile	App runs as `appuser`, not root
Resource Limits	Docker Compose	1GB memory, 1 CPU core max
Capability Dropping	Docker Compose	All capabilities dropped except NET_BIND_SERVICE
Nginx Headers	nginx.conf	X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy

10. Health Monitoring

Health Check Endpoint

Available at `GET /health`, returns JSON with:

- `status`: "ok" or "degraded"
- `timestamp`: Current server time
- `uptime`: Seconds since server start
- `memoryUsage`: RSS, heap total, heap used
- `cpuUsage`: User and system CPU time
- `environment`: "development" or "production"
- `version`: App version from package.json

Auto-degrades to `"degraded"` if essential directories are missing or memory usage exceeds 80%.

Docker Health Checks

Both the app container and Nginx container have built-in health checks that run every 30 seconds. Docker will automatically restart unhealthy containers.

Recommended Monitoring Tools

- Uptime Robot (free tier available)
- Pingdom
- Prometheus + Grafana
- Nagios

11. Template Customization

The resume and cover letter templates are defined in `lib/templates.js`.

Resume Template

- Single-column layout (ATS-optimal)
- Standard fonts: Calibri, Arial, Helvetica Neue
- Sections: Header (name + contact), Summary, Experience, Education, Skills, Projects, Certifications
- Clean section dividers with uppercase titles
- Bullet points for experience descriptions

Cover Letter Template

- Same font family as resume
- Sender info block at top
- Auto-generated date
- Paragraph-based body
- Signature line at bottom

PDF Generation

PDFs are generated with Puppeteer (headless Chromium). Configuration in `lib/pdf.js`:

- Format: A4
- Margins: 0.4in on all sides
- Background rendering enabled
- Runs with `--no-sandbox` flag (Docker-compatible)

12. Troubleshooting

Problem	Solution
Port 3000 already in use	Change `PORT` in `.env`, or find and stop the process using that port: `lsof -i :3000` (Linux/Mac) or `netstat -ano | findstr :3000` (Windows)
"Failed to fetch job description" (403)	Many sites (LinkedIn, Indeed) block scraping. Use the "Paste Job Description" textarea instead of the URL field.
PDF generation fails	Ensure Puppeteer/Chromium is installed. In Docker, the Dockerfile handles this. Locally, run `npx puppeteer install`.
"Cannot GET /"	Server may not be running. Check `node server.js` output for errors.
Docker permission issues	Fix with: `chmod -R 755 uploads output`
Nginx 502 Bad Gateway	The app container may be down. Check: `docker-compose ps` and `docker-compose logs app`
AI generation fails	Verify your `GROQ_API_KEY` is valid and has quota. Check Groq dashboard at `console.groq.com`.
File upload fails	Max file size is 10MB. Supported formats: PDF, DOCX, TXT, MD, RTF, DOC.
Session data lost	Sessions are in-memory by default. Restarting the server clears all sessions. This is by design.

> **Important:** Session data (generated resumes) is stored in-memory. It is lost when the server restarts. This is a privacy feature — no user data is persisted permanently.
